# Objects and Classes – Part 3

# Definitions we learnt so far.

- Object
- Class
- Abstraction
- Mutators **(setters)**
- Accessors **(getters)**
- Private Helpers
- Initialization
- Constructor
- Default constructor
- Constructor overloading
- References
- Multiple object references
- ‘this’ implicit parameter
- Wrapper classes vs primitive classes
- Autoboxing vs unboxing
- Test bench
- Regression testing
- Unit testing

# Test bench

A ***testbench*** is a program whose job is to thoroughly test another program (or portion) via a series of input/output checks known as ***test cases***.

***Unit testing*** means to create and run a testbench for a specific item (or "unit") like a method or a class.

### Regression testing

***Regression testing*** means to retest an item like a class anytime that item is changed; if previously-passed test cases fail, the item has "regressed".
A testbench should be maintained along with the item, to always be usable for regression testing.

# Test bench (Example)

Class to test: StatsInfo.java

```java
public class StatsInfo {

   // Note: This class intentionally has errors

   private int num1;
   private int num2;

   public void setNum1(int numVal) {
      num1 = numVal;
   }

   public void setNum2(int numVal) {
      num2 = numVal;
   }

   public int getNum1() {
      return num1;
   }

   public int getNum2() {
      return num1;
   }

   public int getAverage() {
      return num1 + num2 / 2;
   }
}
```

Testbench: StatsInfoTest.java

```java
public class StatsInfoTest {
   public static void main(String[] args) {
      StatsInfo testData = new StatsInfo();

      // Typical testbench tests more thoroughly

      System.out.println("Beginning tests.");

      // Check set/get num1
      testData.setNum1(100);
      if (testData.getNum1() != 100) {
         System.out.println("   FAILED set/get num1");
      }

      // Check set/get num2
      testData.setNum2(50);
      if (testData.getNum2() != 50) {
         System.out.println("   FAILED set/get num2");
      }

      // Check getAverage()
      testData.setNum1(10);
      testData.setNum2(20);
      if (testData.getAverage() != 15) {
         System.out.println("   FAILED GetAverage for 10, 20");
      }

      testData.setNum1(-10);
      testData.setNum2(0);
      if (testData.getAverage() != -5) {
         System.out.println("   FAILED GetAverage for -10, 0");
      }

      System.out.println("Tests complete.");
   }
}
```

A testbench (also has a main method) is different from the main program.

The values are hardcoded in a testbench to see if all the methods work as designed.

# Packages

A package in Java is used to group related classes. Think of it as **a folder in a file directory**. We use packages to avoid name conflicts, and to write a better maintainable code. Packages are divided into two categories:

- Built-in Packages (packages from the Java API)
- User-defined Packages (create your own packages)

# Build-in packages

- The Java API is a library of prewritten classes, that are free to use, included in the Java Development Environment.
- The library contains components for managing input, database programming, and much much more. The complete list can be found at Oracles website: https://docs.oracle.com/javase/8/docs/api/.

Table 7.20.1: Common Java packages.

| Package | Sample package members | Description |
|---|---|---|
| ***java.lang*** | String, Integer, Double, Math | Contains fundamental Java classes. Automatically imported by Java. |
| ***java.util*** | Collection, ArrayList, LinkedList, Scanner | Contains the Java collections framework classes and miscellaneous utility classes. |
| ***java.io*** | File, InputStream, OutputStream | Contains classes for system input and output. |
| ***javax.swing*** | JFrame, JTextField, JButton | Contains classes for building graphical user interfaces. |

# Importing a package

## Syntax

```
import package.name.Class;   // Import a single class
import package.name.*;   // Import the whole package
```

Example:

```
import java.util.Scanner;
class MyClass {
public static void main(String[] args) {
                Scanner myObj = new Scanner(System.in);
                String userName = myObj.nextLine();
System.out.println("Username is: " + userName);
}
}
```

# User-Defined Package

To create your own package, you need to understand that Java uses a file system directory to store them. Just like folders on your computer:

```
package mypack;
class MyPackageClass {
  public static void main(String[] args) {
    System.out.println("This is my package!");
  }
}
```

# Java ArrayList

The ArrayList class is a resizable array, which can be found in the java.util package.

The difference between a built-in array and an ArrayList in Java, is that the size of an array cannot be modified (if you want to add or remove elements to/from an array, you have to create a new one). While elements can be added and removed from an ArrayList whenever you want. The syntax is also slightly different:

```
import java.util.ArrayList; // import the ArrayList class

ArrayList<String> cars = new ArrayList<String>(); // Create an ArrayList object
```

# ArrayList Methods

| | | |
|---|---|---|
| ***add()*** | `add(element)`<br>Create space for and add the element at the end of the list. | `// List originally empty`<br>`valsList.add(31); // List now: 31`<br>`valsList.add(41); // List now: 31 41` |
| ***get()*** | `get(index)`<br>Returns the element at the specified list location known as the ***index***.<br>Indices start at 0. | `// List originally: 31 41 59. Assume x is an int.`<br>`x = valsList.get(0);  // Assigns 31 to x`<br>`x = valsList.get(1);  // Assigns 41`<br>`x = valsList.get(2);  // Assigns 59`<br>`x = valsList.get(3);  // Error: No such element` |
| ***set()*** | `set(index, element)`<br>Replaces the element at the specified position in this list with the specified element. | `// List originally: 31 41 59`<br>`valsList.set(1, 119);  // List now 31 119 59` |
| ***size()*** | `size()`<br>Returns the number of list elements. | `// List originally: 31 41 59. Assume x is an int.`<br>`x = valsList.size();  // Assigns x with 3` |

# Classes and ArrayLists

A programmer commonly uses classes and ArrayLists together

- A class can have an ArrayList
- An array list can access a class

| | | |
|---|---|---|
| ***size()*** | `int size()`<br><br>Returns the number of elements in the ArrayList. | `if (teamNums.size() > 0) { // Size is 5 so condition is true`<br>`  ...`<br>`}` |
| ***isEmpty()*** | `boolean isEmpty()`<br><br>Returns true if the ArrayList does not contain any elements. Otherwise, returns false. | `if (teamNums.isEmpty()) { // Size is 5 so condition is false`<br>`  ...`<br>`}` |
| ***clear()*** | `void clear()`<br><br>Removes all elements from the ArrayList. | `teamNums.clear();  // ArrayList now has no elements`<br>`System.out.println(teamNums.size()); // Prints 0` |

# Linked List

Linked List is identical to ArrayList.

## How the ArrayList works

The `ArrayList` class has a regular array inside it. When an element is added, it is placed into the array. If the array is not big enough, a new, larger array is created to replace the old one and the old one is removed.

## How the LinkedList works

The `LinkedList` stores its items in "containers." The list has a link to the first container and each container has a link to the next container in the list. To add an element to the list, the element is placed into a new container and that container is linked to one of the other containers in the list.

## When To Use

Use an `ArrayList` for storing and accessing data, and `LinkedList` to manipulate data.

# Linked List Methods

| Method | Description |
| --- | --- |
| addFirst() | Adds an item to the beginning of the list. |
| addLast() | Add an item to the end of the list |
| removeFirst() | Remove an item from the beginning of the list. |
| removeLast() | Remove an item from the end of the list |
| getFirst() | Get the item at the beginning of the list |
| getLast() | Get the item at the end of the list |

# Java Documentation for Classes

The ***Javadoc*** tool parses source code along with specially formatted comments to generate documentation.

| Block tag | Compatibility | Description |
|---|---|---|
| @author | classes | Used to specify an author. |
| @version | classes | Used to specify a version number. |
| @param | methods, constructors | Used to describe a parameter. |
| @return | methods | Used to describe the value or object returned by the method. |
| @see | all | Used to refer reader to relevant websites or class members. |

# Static Fields and Methods

### Static fields

The keyword ***static*** indicates a variable is allocated in memory only once during a program's execution. Static variables reside in the program's static memory region and have a global scope. Thus, static variables can be accessed from anywhere in a program.

### Static member methods

A ***static member method*** is a class method that is independent of class objects. Static member methods are typically used to access and mutate private static fields from outside the class.

# In class activity - Sorting an ArrayList

```
import java.util.ArrayList;
import java.util.Scanner;

public static void main(String [] args) {
     ArrayList<Integer> numbers = new ArrayList<Integer>();
     System.out.println("Enter numbers one after another. Once done enter -1");

    \\  Keep scanning from user until user enters -1
     \\ Store the numbers in ascending order as the user enters
              \\ have a loop to check if the number where the number should be and insert it in
                 that index
     \\ Print out the final array.

}
```

SORTING HAS TO HAPPEN IN REAL TIME as user inputs. DO NOT USE COLLECTIONS.SORT()

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {
        int flag = 0;
        Scanner sc = new Scanner(System.in);
        ArrayList<Integer> numbers = new ArrayList<Integer>();
        Integer x = sc.nextInt();           // scan the very first number
        while (!x.equals(-1)) {             // if the first number is -1, while loop does not get execute
            if (numbers.size() == 0){       // this will get executed just once
                numbers.add(x);             // very first element gets added to ArrayList
                x=sc.nextInt();             // second element gets scanned here
                continue;                   // remaining of the while loop is not executed
            }
            for (int j = 0; j < numbers.size(); j++) {  // for loop to iterate through ArrayList
                if (x < numbers.get(j)) {
                    numbers.add(j,x);                   // TRUE if new element is < existing element
                    flag = 1;
                    break;                              // will break as long as at least one element ...
                }                                       //   ..... is larger than the new element
            }
            if (flag == 0){
                numbers.add(x);                         // will execute if new element is largest
            }
            x=sc.nextInt();                             // scan new element
            flag = 0;                                   // should reset the flag for next iteration
        }
        System.out.println(numbers);
    }
}
```